

Dynamic Bonding Curve

0.1.10

Smart Contract Security Assessment

March 2026

Prepared for:

Meteora

Prepared by:

Offside Labs

Sirius Xie

Ronny Xing





Contents

1	About Offside Labs	2
2	Executive Summary	3
3	Summary of Findings	5
4	Key Findings and Recommendations	6
4.1	Unbounded locked_vesting cliff can permanently brick migration	6
4.2	Informational and Undetermined Issues	8
5	Disclaimer	9



1 About Offside Labs

Offside Labs is a leading security research team, composed of top talented hackers from both academia and industry.

We possess a wide range of expertise in modern software systems, including, but not limited to, *browsers, operating systems, IoT devices, and hypervisors*. We are also at the forefront of innovative areas like *cryptocurrencies* and *blockchain technologies*. Among our notable accomplishments are remote jailbreaks of devices such as the **iPhone** and **PlayStation 4**, and addressing critical vulnerabilities in the **Tron Network**.

Our team actively engages with and contributes to the security community. Having won and also co-organized *DEFCON CTF*, the most famous CTF competition in the Web2 era, we also triumphed in the **Paradigm CTF 2023** within the Web3 space. In addition, our efforts in responsibly disclosing numerous vulnerabilities to leading tech companies, such as *Apple, Google, and Microsoft*, have protected digital assets valued at over **\$300 million**.

In the transition towards Web3, Offside Labs has achieved remarkable success. We have earned over **\$9 million** in bug bounties, and **three** of our innovative techniques were recognized among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.



<https://offside.io/>



<https://github.com/offsidelabs>



https://twitter.com/offside_labs



2 Executive Summary

Introduction

Offside Labs completed a security audit of *Dynamic Bonding Curve* smart contracts, starting on March 6th, 2026, and concluding on March 12th, 2026.

Project Overview

Meteora Dynamic Bonding Curve Release 0.1.10 introduces a new protocol authorization model, replacing the previous single purpose fee operator with a more general Operator account abstraction and adding support for protocol fee zap. The release also refines the migration framework by consolidating migration amount calculation, migration fee calculation, and initial liquidity handling, strengthening the DAMM v2 migration flow and adding support for compounding mode. In addition, the configuration layer is extended with migrated pool parameters and validation, particularly for DAMM v2 fee and locking scenarios, with corresponding updates to error handling, utility modules, dependencies, and test coverage.

Audit Scope

The assessment scope contains mainly the smart contracts of the dynamic-bonding-curve program for the *Dynamic Bonding Curve* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- Dynamic Bonding Curve
 - Codebase: <https://github.com/MeteoraAg/dynamic-bonding-curve>
 - Release 0.1.10: [PR-174](#)
 - Commit Hash: 93fd1a18ad65b9b4c961786d223341024ce9ad9a

The issues described in this report have been resolved by the following commit:

- Dynamic Bonding Curve
 - Codebase: <https://github.com/MeteoraAg/dynamic-bonding-curve>
 - Commit Hash: 4dacc07f1187f7a9eda06896faa29aaebd104a5e

We listed the files we have audited below:

- Dynamic Bonding Curve
 - programs/dynamic-bonding-curve/src/access_control.rs
 - programs/dynamic-bonding-curve/src/constants.rs
 - programs/dynamic-bonding-curve/src/curve.rs
 - programs/dynamic-bonding-curve/src/error.rs
 - programs/dynamic-bonding-curve/src/instructions/admin/ix_close_operator_account.rs
 - programs/dynamic-bonding-curve/src/instructions/admin/ix_create_operator_account.rs



- programs/dynamic-bonding-curve/src/instructions/admin/mod.rs
- programs/dynamic-bonding-curve/src/instructions/migration/dynamic_amm_v2/migrate_damm_v2_initialize_pool.rs
- programs/dynamic-bonding-curve/src/instructions/migration/dynamic_amm_v2/mod.rs
- programs/dynamic-bonding-curve/src/instructions/migration/meteora_damm/migrate_meteora_damm_initialize_pool.rs
- programs/dynamic-bonding-curve/src/instructions/operator/ix_claim_protocol_fee.rs
- programs/dynamic-bonding-curve/src/instructions/operator/ix_claim_protocol_pool_creation_fee.rs
- programs/dynamic-bonding-curve/src/instructions/operator/ix_zap_protocol_fee.rs
- programs/dynamic-bonding-curve/src/instructions/operator/mod.rs
- programs/dynamic-bonding-curve/src/instructions/partner/ix_create_config.rs
- programs/dynamic-bonding-curve/src/lib.rs
- programs/dynamic-bonding-curve/src/math/safe_math.rs
- programs/dynamic-bonding-curve/src/migration_handler/compounding_liquidity.rs
- programs/dynamic-bonding-curve/src/migration_handler/concentrated_liquidity.rs
- programs/dynamic-bonding-curve/src/migration_handler/mod.rs
- programs/dynamic-bonding-curve/src/params/liquidity_distribution.rs
- programs/dynamic-bonding-curve/src/state/config.rs
- programs/dynamic-bonding-curve/src/state/mod.rs
- programs/dynamic-bonding-curve/src/state/operator.rs
- programs/dynamic-bonding-curve/src/state/virtual_pool.rs
- programs/dynamic-bonding-curve/src/utils/damm_v2_utils.rs
- programs/dynamic-bonding-curve/src/utils/mod.rs
- programs/dynamic-bonding-curve/src/utils/token.rs

Findings

The security audit revealed:

- 0 critical issue
- 0 high issue
- 1 medium issues
- 0 low issue
- 1 informational issues

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.



3 Summary of Findings

ID	Title	Severity	Status
01	Unbounded locked_vesting cliff can permanently brick migration	Medium	Fixed
02	Dynamic Fee Enable Path Can Silently Fall Back to None	Informational	Fixed



4 Key Findings and Recommendations

4.1 Unbounded locked_vesting cliff can permanently brick migration

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Data Validation

Description

For pools with `LockedVesting` enabled, the following check is performed in `LockedVestingParams::validate` during `create_config`:

```
300     pub fn validate(&self) -> Result<()> {
301         if self.has_vesting() {
302             let total_amount = self.get_total_amount()?;
303             require!(
304                 self.frequency != 0 && total_amount != 0,
305                 PoolError::InvalidVestingParameters
306             );
307         }
308         Ok(())
309     }
```

[programs/dynamic-bonding-curve/src/instructions/partner/ix_create_config.rs#L300-L309](#)

However, there is no upper-bound check on `cliff_duration_from_migration_time` here.

After the pool completes its final swap and enters `PostBondingCurve`, the next step is to invoke the `create_locker` IX, where the locker parameters are constructed:

```
272     pub fn to_create_vesting_escrow_params(
273         &self,
274         finish_curve_timestamp: u64,
275     ) -> Result<CreateVestingEscrowParameters> {
276         let cliff_time =
277             finish_curve_timestamp.safe_add(self.cliff_duration_from_migration_time)?;
```

[programs/dynamic-bonding-curve/src/instructions/partner/ix_create_config.rs#L272-L277](#)

If `LockedVestingParams.cliff_duration_from_migration_time` is set to a large value (for example, `u64::MAX`), then the `safe_add` operation may fail directly due to arithmetic overflow, causing the `create_locker` IX to fail, and preventing this pool's migration progress from moving to the next `LockedVesting` stage.



Impact

This kind of invalid config can cause all pools created through this config to become permanently unable to complete migration after the curve is completed. At that point, swaps can no longer continue in the pool, and migrated pools in DAMM and DAMM v2 also cannot be created. The tokens will remain stuck in the DBC pool and cannot be transferred out.

Proof of Concept

1. A partner creates a config with `locked_vesting` enabled and sets `cliff_duration_from_migration_time` to a very large value.
2. `create_config` completes successfully because this field has no upper-bound validation.
3. The creator uses this config to create a pool, and initialization also succeeds, because initialization only uses `locked_vesting.get_total_amount()` and does not use `cliff_duration_from_migration_time`.
4. Users perform swaps normally, and the curve completes normally.
5. The pool's migration progress switch into `PostBondingCurve`.
6. When anyone invokes `create_locker`, `finish_curve_timestamp.safe_add(cliff_duration_from_migration_time)` overflows, and the transaction fails and returns immediately.
7. The pool remains permanently stuck in `PostBondingCurve`, and subsequent `migrate_meteora_damm` and `migrate_damm_v2` can no longer be executed.

Recommendation

It is recommended to add a check for `LockedVestingParams.cliff_duration_from_migration_time` in `LockedVestingParams::validate()` of the `create_config` IX, to ensure that the result of `finish_curve_timestamp + cliff_duration_from_migration_time` cannot overflow.

Mitigation Review Log

Fixed in the commit `4dacc07f1187f7a9eda06896faa29aaebd104a5e`.



4.2 Informational and Undetermined Issues

Dynamic Fee Enable Path Can Silently Fall Back to None

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Code QA

`build_damm_v2_dynamic_fee_params` uses `Ok(calculate_dynamic_fee_params(min_base_fee_numerator).ok())` in the `DammV2DynamicFee::Enable` branch, so if `calculate_dynamic_fee_params` returns `Err`, it will be silently converted into `Ok(None)`.

It is recommended to propagate the error explicitly instead, for example by using `Ok(Some(calculate_dynamic_fee_params(min_base_fee_numerator)?))`, so that enabling dynamic fee cannot silently degrade into `None` when the internal calculation fails.



5 Disclaimer

This audit report is provided for informational purposes only and is not intended to be used as investment advice. While we strive to thoroughly review and analyze the smart contracts in question, we must clarify that our services do not encompass an exhaustive security examination. Our audit aims to identify potential security vulnerabilities to the best of our ability, but it does not serve as a guarantee that the smart contracts are completely free from security risks.

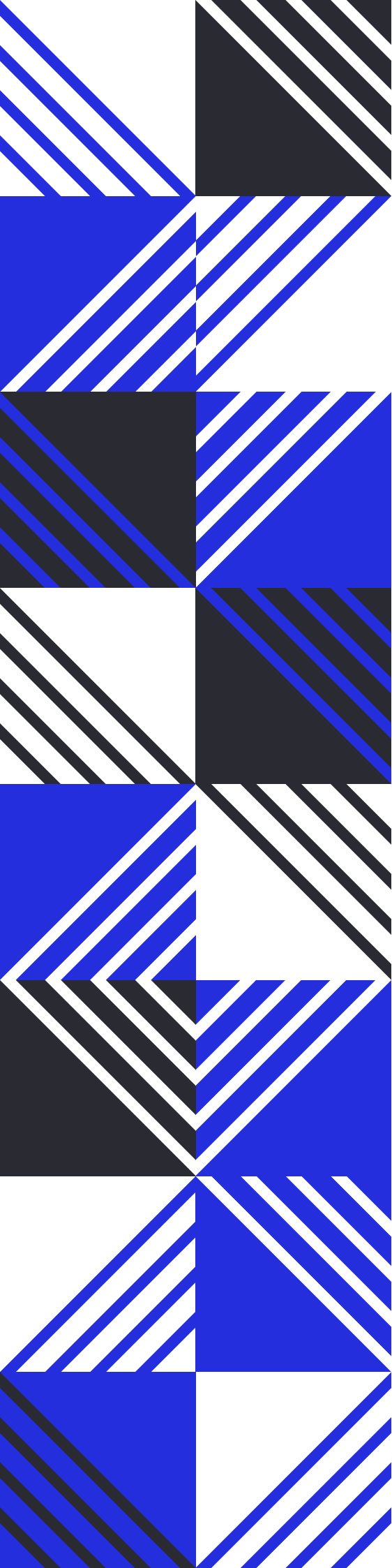
We expressly disclaim any liability for any losses or damages arising from the use of this report or from any security breaches that may occur in the future. We also recommend that our clients engage in multiple independent audits and establish a public bug bounty program as additional measures to bolster the security of their smart contracts.

It is important to note that the scope of our audit is limited to the areas outlined within our engagement and does not include every possible risk or vulnerability. Continuous security practices, including regular audits and monitoring, are essential for maintaining the security of smart contracts over time.

Please note: we are not liable for any security issues stemming from developer errors or misconfigurations at the time of contract deployment; we do not assume responsibility for any centralized governance risks within the project; we are not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By using this report, the client acknowledges the inherent limitations of the audit process and agrees that our firm shall not be held liable for any incidents that may occur subsequent to our engagement.

This report is considered null and void if the report (or any portion thereof) is altered in any manner.



 <https://offside.io/>

 <https://github.com/offsidelabs>

 https://twitter.com/offside_labs